Services et SharedFlow sous android

Plan

I. SERVICES ---- 2

I.1. QU'EST-CE QU'UN SERVICE ? ---- 2
I.2. EXEMPLES ---- 2
I.3. LES TYPES DES SERVICES ---- 2
I.4. CYCLE DE VIE D'UN SERVICE ---- 2
I.5. LANCER UN SERVICE ---- 3
I.5.1. Lancer un service local ---- 3
I.5.2. Lancer un service distant ---- 3
I.6. ARRETER UN SERVICE ---- 3
I.7. AJOUT D'UN SERVICE A UNE APPLICATION ---- 3
I.7.1. La classe Service ---- 3
I.7.2. La classe IntentService ---- 3
I.7.3. Déclarer le service dans le AndroidManifest.xml ---- 3

II. COMMUNICATION MODERNE PAR SHAREDFLOW ---- 4

II.1. QU'EST-CE QU'UN SHAREDFLOW SOUS FORME D'EVENTBUS ? ---- 4
II.2. DEFINITION DU BUS DE DONNEES (`TIMEEVENTBUS`) ---- 4
II.3. GENERER LA DONNEE A DIFFUSER ---- 4
II.4. DIFFUSER L'EVENEMENT ---- 4
II.5. S'ENREGISTRER A L'ECOUTE DES DIFFUSIONS (LA COLLECTE) ---- 4
II.6. SE DESENREGISTRER DE L'ECOUTE ---- 5
II.7. EXEMPLE D'ENVOI DE PLUSIEURS VALEURS(X, Y ET Z) ---- 5
II.7.1. Déclaration de l'objet `AccelerometerEventBus` ---- 5
II.7.2. Diffusion de l'évènement ---- 5
II.7.3. Enregistrement à l'écoute des diffusions (La Collecte) ---- 5

III. APPLICATION ---- 6

III.1. ENONCE ---- 6
III.2. SOLUTION ---- 6
III.2.1. L'objet TimeEventBus ---- 6
III.2.2. La classe MainActivity ---- 6
III.2.3. La classe TempsSrvice ---- 7

I. Services

I.1. Qu'est-ce qu'un service ?

Tout comme les activités, les services possèdent un cycle de vie ainsi qu'un contexte qui contient des informations spécifiques sur l'application et qui constitue une interface de communication avec le restant du système. Ainsi, on peut dire que les services sont des composants très proches des activités (et beaucoup moins des receivers, qui eux ne possèdent pas de contexte). Cette configuration leur prodigue la même grande flexibilité que les activités. En revanche, à l'opposé des activités, les services ne possèdent pas d'interface graphique : c'est pourquoi on les utilise pour effectuer des travaux d'arrière-plan [1].

I.2. Exemples

Un exemple typique est celui du lecteur de musique. Vous laissez à l'utilisateur l'opportunité de choisir une chanson à l'aide d'une interface graphique dans une activité, puis il est possible de manipuler la chanson dans une seconde activité qui nous montre un lecteur avec des commandes pour modifier le volume ou mettre en pause. Mais si l'utilisateur veut regarder une page web en écoutant la musique ? Comme une activité a besoin d'afficher une interface graphique, il est impossible que l'utilisateur regarde autre chose que le lecteur quand il écoute la musique. On pourrait éventuellement envisager de passer par un receiver, mais celui-ci devrait résoudre son exécution en dix secondes, ce n'est donc pas l'idéal pour un lecteur. La solution la plus évidente est bien sûr de faire jouer la musique par un service, comme ça votre client pourra utiliser une autre application sans pour autant que la musique s'interrompe. Un autre exemple est celui du lecteur d'e-mails qui va vérifier ponctuellement si vous avez reçu un nouvel e-mail [1].

I.3. Les types des services

Il existe deux types de services [1]:

Les plus courants sont les services *locaux* (on trouve aussi le terme *started* ou *unbound service*), où l'activité qui lance le service et le service en lui-même appartiennent à la même application.

Il est aussi possible qu'un service soit lancé par un composant qui appartient à une autre application, auquel cas il s'agit d'un service *distant* (on trouve aussi le terme bound *service*). Dans ce cas de figure, il existe toujours une interface qui permet la communication entre le processus qui a appelé le service et le processus dans lequel s'exécute le service. Cette communication permet d'envoyer des requêtes ou récupérer des résultats par exemple. Le fait de communiquer entre plusieurs processus s'appelle l'IPC. Il peut bien sûr y avoir plusieurs clients liés à un service.

Il est aussi possible que le service expérimente les deux statuts à la fois. Ainsi, on peut lancer un service local et lui permettre d'accepter les connexions distantes par la suite.

I.4. Cycle de vie d'un service

De manière analogue aux activités, les services traversent plusieurs étapes pendant leur vie et la transition entre ces étapes est matérialisée par des méthodes de *callback*. Cependant, le cycle des services contient beaucoup moins d'étapes. La figure suivante est un schéma qui résume ce fonctionnement [1].

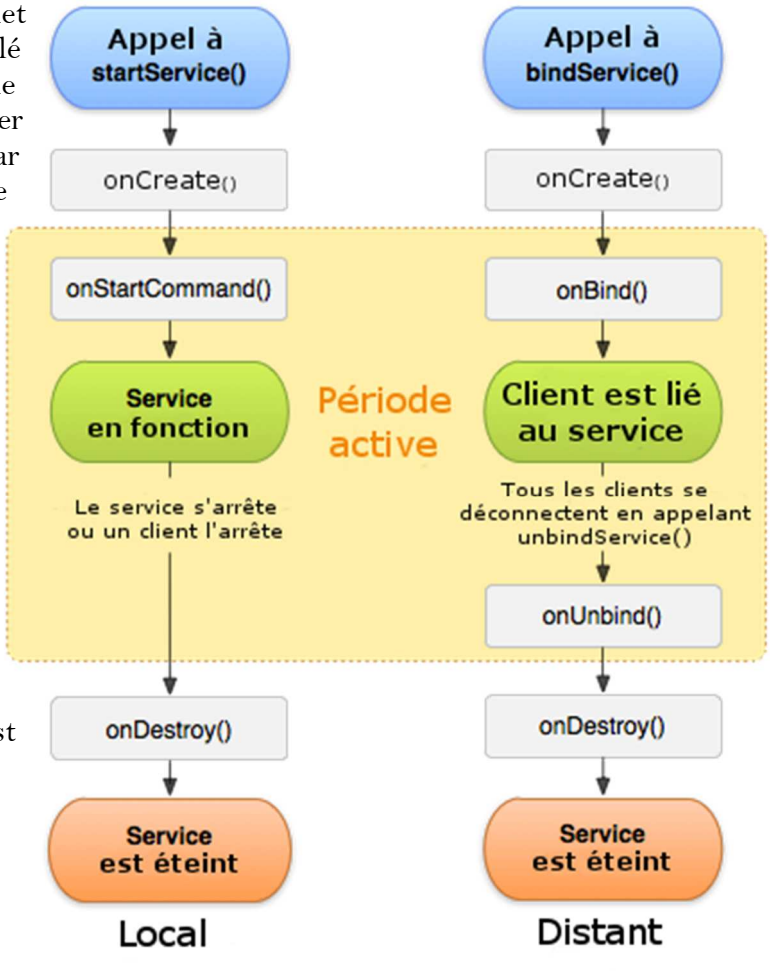


Figure 1 : cycle de vie d'un service local et d'un service distant [1].

I.5. Lancer un service

I.5.1. Lancer un service local

Un service local est lancé à partir d'une activité avec la méthode **ComponentName startService(Intent service)**. La variable retournée donne le package accompagné du nom du composant qui vient d'être lancé.

```
Exemple :
val intent = Intent(this, UnService::class.java)
startService(intent)
```

I.5.2. Lancer un service distant

Un service distant est lancé à partir d'une activité avec la méthode boolean bindService(Intent service, ServiceConnection conn, int flags).

I.6. Arrêter un service

Un service local est lancé à partir d'une activité avec la méthode boolean stopService (Intent service).

```
Exemple :
val intent = Intent(this, UnService::class.java)
stopService(intent)
```

I.7. Ajout d'un Service à une application

I.7.1. La classe Service

La classe Service du package android.app permet de déclarer un service de base. Le code sera alors exécuté dans le thread principal, alors ce sera au développeur de créer un nouveau thread pour ne pas perturber l'exécution du thread UI [1].

Service est une classe abstraite qui contient une seule méthode abstraite qui est **IBinder onBind(Intent intent).**

```
Exemple :
package com.serv

import android.app.Service
import android.content.Intent
import android.os.IBinder

class MonService : Service() {
    override fun onBind(intent: Intent?): IBinder? {
        return null
    }
}
```

I.7.2. La classe IntentService

La classe IntentService du package android.app est une sous classe de la classe Service. Cette classe va créer elle-même un thread et gérer les requêtes que vous lui enverrez dans une file. À chaque fois que vous utiliserez startService() pour lancer ce service, la requête sera ajoutée à la file et tous les éléments de la file seront traités par ordre d'arrivée. Le service s'arrêtera dès que la file sera vide [1].

I.7.3. Déclarer le service dans le AndroidManifest.xml

Utiliser la balise <service> et donner dans android :name le nom du package et de la classe du service.

```
Exemple :
<service android:name=" com.serv.MonService"

</service>
```

II. Communication moderne par SharedFlow

II.1. Qu'est-ce qu'un SharedFlow sous forme d'EventBus ?

- Pour pouvoir diffuser des événements à travers toute l'application sans dépendre du système Android (contrairement aux `BroadcastReceivers`), Kotlin propose les **`SharedFlow`**.
- Un `SharedFlow` est un flux chaud (*hot stream*) capable d'émettre des valeurs à destination de un ou plusieurs abonnés.
- L'utilisation d'un objet unique (**Singleton**) permet de centraliser la communication (Pattern *Event Bus*) de manière propre et performante.
- Grâce au paramètre de configuration `replay = 1`, tout nouvel abonné récupère immédiatement la dernière valeur enregistrée.

II.2. Définition du Bus de données (`TimeEventBus`)

On crée un objet global `object` qui encapsule le `MutableSharedFlow` en privé pour des raisons de sécurité, tout en exposant une version en lecture seule et une fonction de diffusion explicite.

```
object TimeEventBus {
    // Un SharedFlow avec un buffer (replay = 1) pour que le dernier message soit
conservé pour les nouveaux abonnés
    private val _dateHeureFlow = MutableSharedFlow<String>(replay = 1)
    val dateHeureFlow = _dateHeureFlow.asSharedFlow()

    // Nom explicite pour diffuser la date et l'heure
    fun diffuserDateHeure(dateHeureFormatee: String) {
        _dateHeureFlow.tryEmit(dateHeureFormatee)
    }
}
```

II.3. Générer la donnée à diffuser

La donnée à transmettre n'est plus encapsulée dans un objet `Intent` global du système, mais peut être n'importe quelle variable (ici, une chaîne de caractères formatée représentant l'heure).

```
// Préparation de la date et l'heure actuelle
val formatter = DateTimeFormatter.ofPattern("HH:mm:ss")
val dateHeureActuelle = LocalTime.now().format(formatter)
```

II.4. Diffuser l'événement

Pour envoyer l'information à travers l'application, on appelle simplement la méthode de diffusion du bus :

```
TimeEventBus.diffuserDateHeure(dateHeureActuelle)
```

Dès que cette ligne est exécutée, la valeur est instantanément propagée à tous les composants abonnés.

II.5. S'enregistrer à l'écoute des diffusions (La Collecte)

Pour écouter les diffusions du flux dans une interface utilisateur (`Activity` ou `Fragment`), on utilise la coroutine liée au cycle de vie (`lifecycleScope` et `repeatOnLifecycle`), remplaçant avantageusement l'ancien `registerReceiver`.

```
// Écoute du flux de date et d'heure dans l'Activity
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
           TimeEventBus.dateHeureFlow.collect { dateHeureRecue ->
           // Utilisation de la donnée reçue
            amb.tvTemps.text = dateHeureRecue
       }
    }
      }
```

II.6. Se désenregistrer de l'écoute

- Avec l'utilisation conjointe de `lifecycleScope` et `repeatOnLifecycle`, **le désenregistrement est entièrement automatique**.
- Lorsque l'activité quitte l'écran (passe en arrière-plan ou est détruite), la coroutine de collecte s'arrête d'elle-même, évitant ainsi toute fuite de mémoire (*memory leak*) ou traitement inutile, sans qu'il soit nécessaire d'appeler manuellement un équivalent de `unregisterReceiver`.

II.7. Exemple d'envoi de plusieurs valeurs(x, y et z)

II.7.1. Déclaration de l'objet `AccelerometerEventBus`

```
// Structure pour stocker les 3 axes
data class AccelerometerData(
    val x: Float,
    val y: Float,
    val z: Float
)

object AccelerometerEventBus {
    private val _sensorFlow = MutableSharedFlow<AccelerometerData>(replay = 1)
    val sensorFlow = _sensorFlow.asSharedFlow()

    fun diffuserCoordonnees(data: AccelerometerData) {
        _sensorFlow.tryEmit(data)
    }
}
```

II.7.2. Diffusion de l'évènement

```
if (event?.sensor?.type == Sensor.TYPE_ACCELEROMETER) {
      val x = event.values[0]
      val y = event.values[1]
      val z = event.values[2]

      // Création de l'objet et diffusion via le bus
      val data = AccelerometerData(x, y, z)
      AccelerometerEventBus.diffuserCoordonnees(data)
  }
```

II.7.3. Enregistrement à l'écoute des diffusions (La Collecte)

```
lifecycleScope.launch {
      repeatOnLifecycle(Lifecycle.State.STARTED) {
          AccelerometerEventBus.sensorFlow.collect { data ->
              // Vous disposez ici de x, y et z en temps réel !
              val x = data.x
              val y = data.y
              val z = data.z

              // Exemple d'affichage :
              // myTextView.text = "X: $x\nY: $y\nZ: $z"
          }
      }
  }
```

III. Application

III.1. Enoncé

Donner le code d'une application android qui permet d'afficher le temps système. Cette application utilise un Service qui permet d'obtenir le temps système et de le diffuser. Un SharedFlow permet d'obtenir le temps système et de l'afficher dans l'activité. L'interface de l'activité est la suivante :



III.2. Solution

III.2.1. L'objet TimeEventBus

```
object TimeEventBus {
    // Un SharedFlow avec un buffer (replay = 1) pour que le dernier message soit conservé pour
les nouveaux abonnés
    private val _dateHeureFlow = MutableSharedFlow<String>(replay = 1)
    val dateHeureFlow = _dateHeureFlow.asSharedFlow()

    // Nom plus explicite que "envoyerMessage"
    fun diffuserDateHeure(dateHeureFormatee: String) {
        _dateHeureFlow.tryEmit(dateHeureFormatee)
    }
}
```

III.2.2. La classe MainActivity

```
class MainActivity : AppCompatActivity() {
    private lateinit var amb: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        initApp()
    }
    private fun initApp() {
// 1. Gonfler le layout via le View Binding et définir la vue racine
        amb = ActivityMainBinding.inflate(layoutInflater)
        setContentView(amb.root)
// 2. Initialisation des écouteurs et du receiver
        ajouterEcouteurs()
    }
    private fun ajouterEcouteurs() {
        amb.btnDemarrer.setOnClickListener {            demarrer()        }
        amb.btnArreter.setOnClickListener  {            arreter()        }
        // Écoute du flux de date et d'heure
        lifecycleScope.launch {
            repeatOnLifecycle(Lifecycle.State.STARTED) {
                TimeEventBus.dateHeureFlow.collect { dateHeureRecue ->
                    // Utilisation explicite (ex: affichage dans un TextView)
                    amb.tvTemps.text = dateHeureRecue

                }
            }
        }
    }
    protected fun arreter() {
        val i = Intent(this, TempsService::class.java)
        stopService(i)
    }
    protected fun demarrer() {
        val i = Intent(this, TempsService::class.java)
        startService(i)
    }
}
```

III.2.3. La classe TempsSrvice

```
class TempsService : Service(), CoroutineScope { // Déclare la classe du service en héritant de Service et de CoroutineScope
    // Crée un Job superviseur pour gérer et grouper les coroutines lancées dans ce service
    private val job = SupervisorJob()
// Définit le contexte d'exécution : Dispatchers.Default pour le travail en arrière-plan (CPU-intensive)
// combiné au 'job' pour pouvoir contrôler/annuler facilement toutes les coroutines en même temps.
    override val coroutineContext = Dispatchers.Default + job
    companion object {
        const val ACTION_TEMPS = "com.serv.temps"
    }
    override fun onBind(intent: Intent?): IBinder? {
        Log.i("Service", "onBind")
        return null
    }
    override fun onCreate() {
        super.onCreate()
        Log.i("Service", "onCreate")
// Lancement de la coroutine avec une boucle infinie propre
        launch {
            while (true) {
                afficher()
                delay(1000) // Suspend la coroutine 1 seconde sans bloquer le système
            }
        }
    }
    protected fun afficher() {
        val dateHeureActuelle = DateFormat.format("MM/dd/yy hh:mm:ss",
System.currentTimeMillis()).toString()
        Log.i("", dateHeureActuelle)
        // Appel de la fonction explicite
        TimeEventBus.diffuserDateHeure(dateHeureActuelle)

    }

    override fun onDestroy() {
        Log.i("", "onDestroy")
// En annulant le job, la coroutine (et donc le while(true)) s'arrête instantanément
        job.cancel()
        super.onDestroy()
    }
}
```